

ENSTORE Investigations and Plans

Don Petravick

June 16, 1998

Outline of the Talk

Investigations

- DESY Mass Storage Model

- Enstore Prototype based on DESY Design

- DESY Workshop

Technical Description of Enstore

Comparison of Prototype to Interim Requirements

Interim Plans and Level of Effort

History and Introduction

The October 97 Von Ruden Review recommended:

The committee urges CD to seriously and immediately take a look at existing alternatives without, however, reducing the effort on further evaluating HPSS.

Visit to DESY in December 1997

Goal: learn how DESY handled their MSS needs
Petravick and Bakken

What we learned:

System evolved from OSM

DESY's system

Efficient – no large overheads in using system

Conceptually simple and modest amount of code

In Production

Recovery, retries, fault analysis, priorities– in place

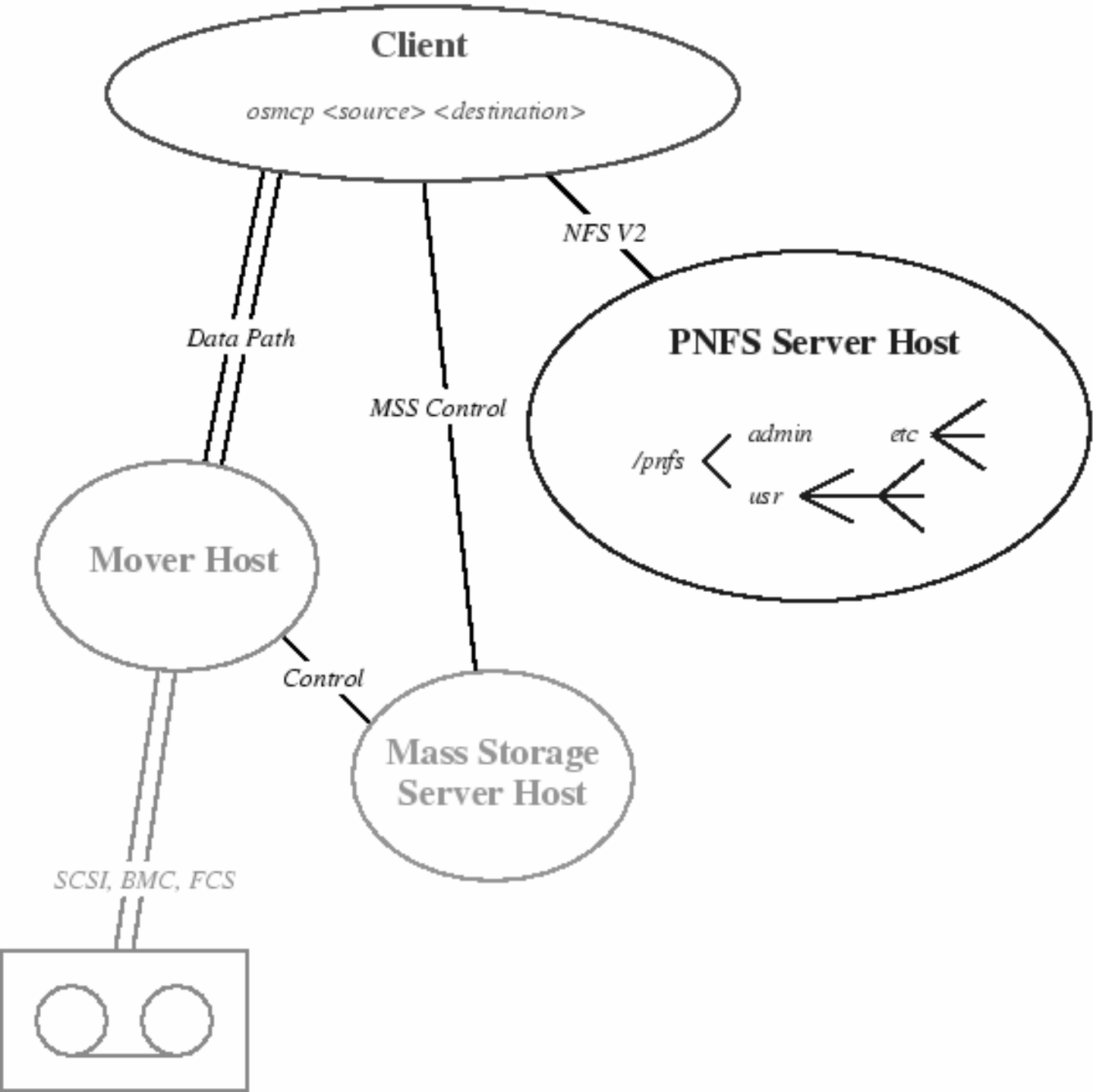
Robotics runs at full speed

Unattended operation, day and night

Uptime > 150 days

Developers in control of MSS software

The OSM Client Connection



How DESY evolved to their present MSS system:

DESY started with commercial OSM System
with source code license

Many local mods to system to satisfy their needs

Some mods – delete part of OSM

no elaborate storage hierarchies

no transparent access

Some mods – additional code necessary to meet
their own requirements

About 40% of original OSM code still in use

Wrote independent namespace package, PNFS

Allows files in MSS system to be named with
unix-like names

Associates info on how to create files in the MSS
system with directories

Distributes administrative information to client
computers

Collaborative Possibilities with DESY:

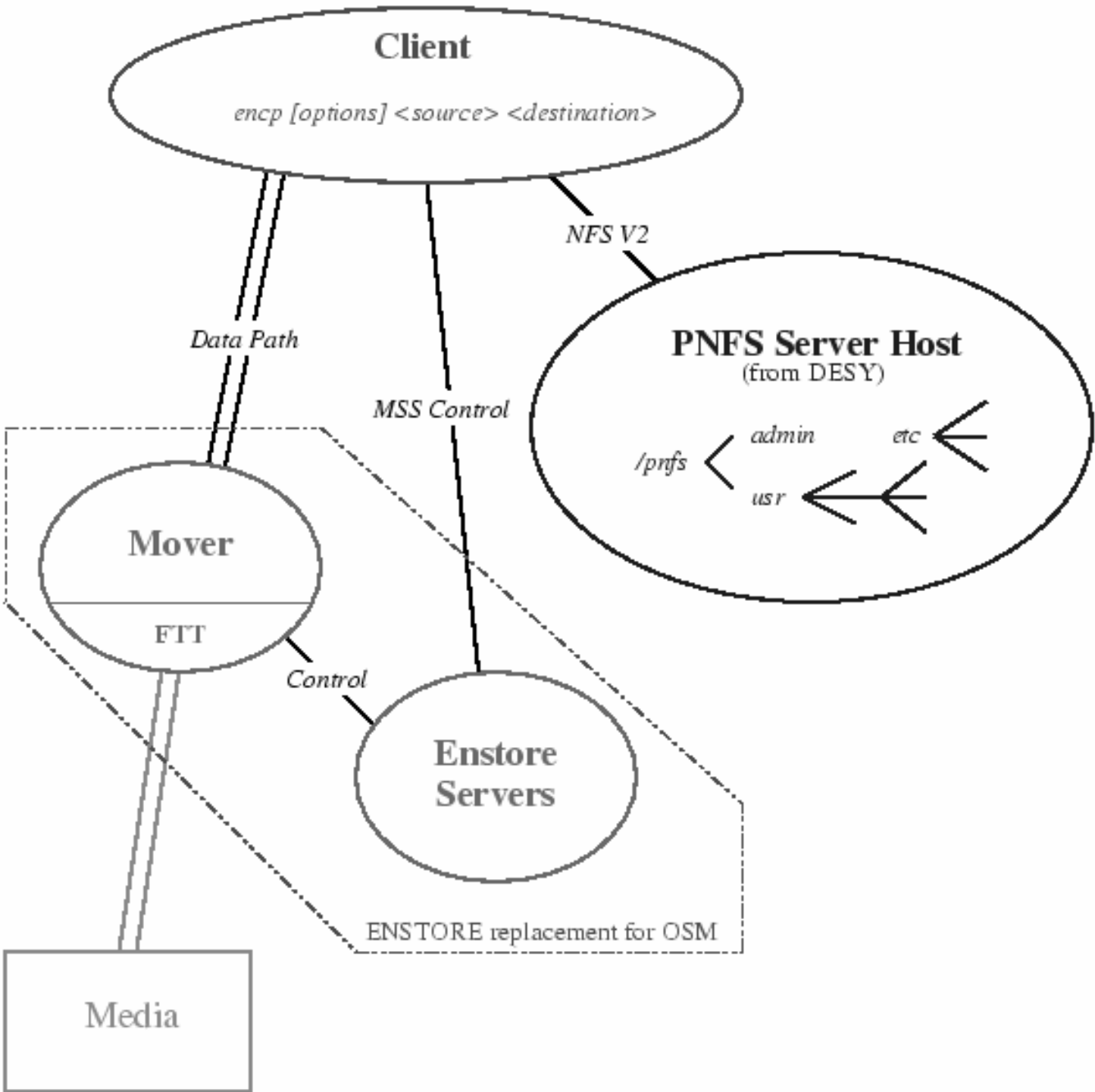
We took back PNFS code from our December visit

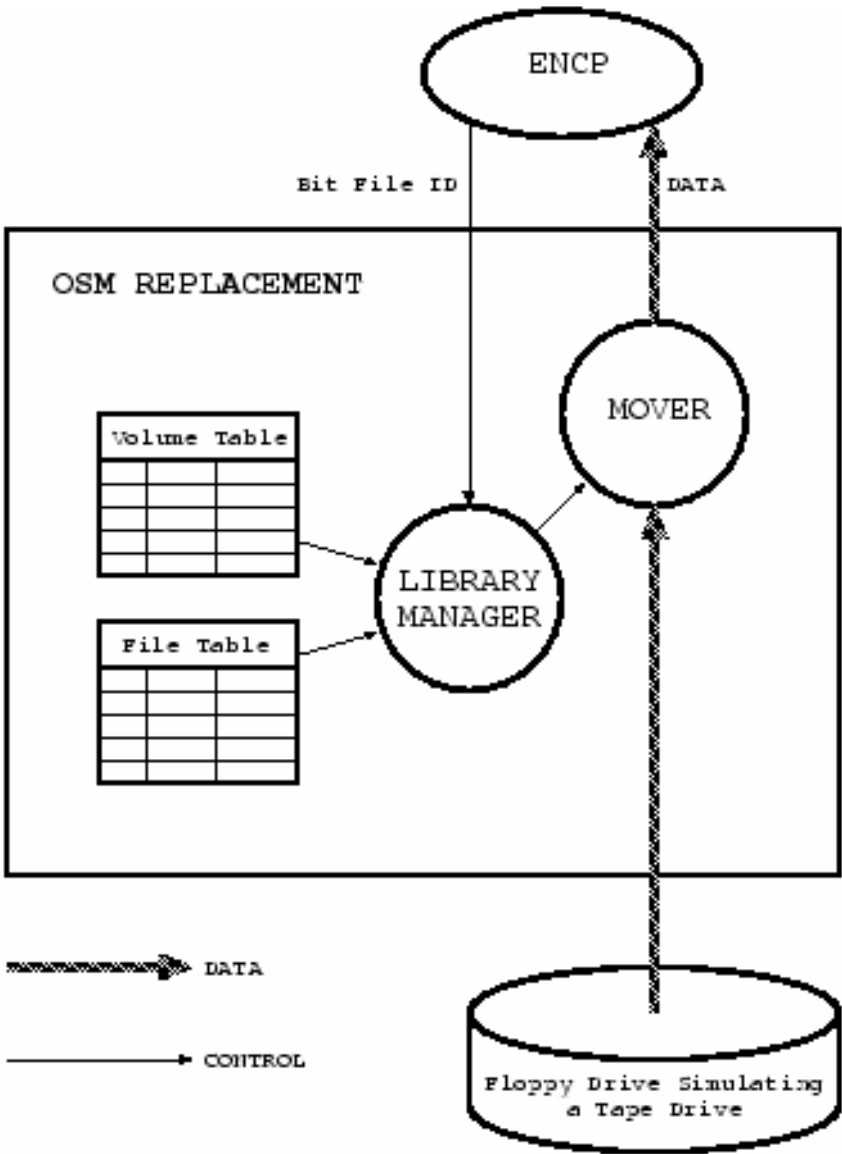
OSM code – can not share because it is licensed
Vendor, CA, shows lack of interest

Spirit of collaboration developed

Possible future sharing in disk cache

Enstore at Fermilab





Essence of the Christmas Prototype

After DESY visit, we wanted to investigate possibility of implementing DESYish system here at Fermilab

Christmas 1997 Early Investigations

Questions:

Did we understand the design thinking that DESY conveyed to us?

How hard would it be to replace OSM parts of DESY system with our own given the fact that DESY is moving away from OSM?

What we did:

Small python-based prototype built

Small 133 MHz Pentium with 16 MB

Distributed TCP design, but ran on single node

Wrote and read floppy disk files instead of tapes

Extended design: added small mover computers

Results:

System ran overnight tests successfully

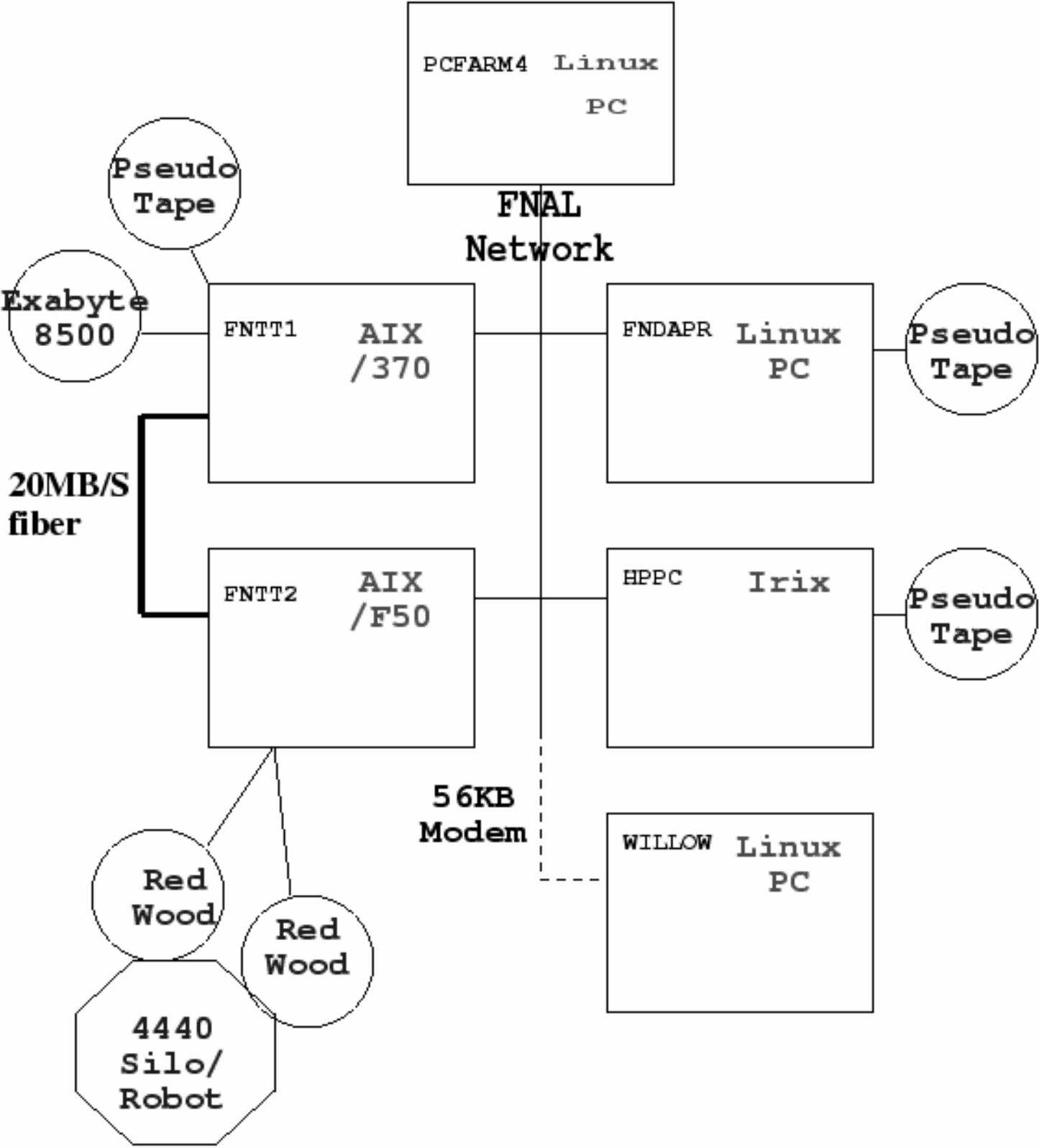
16 concurrent end-user clients

Decided system looked very promising

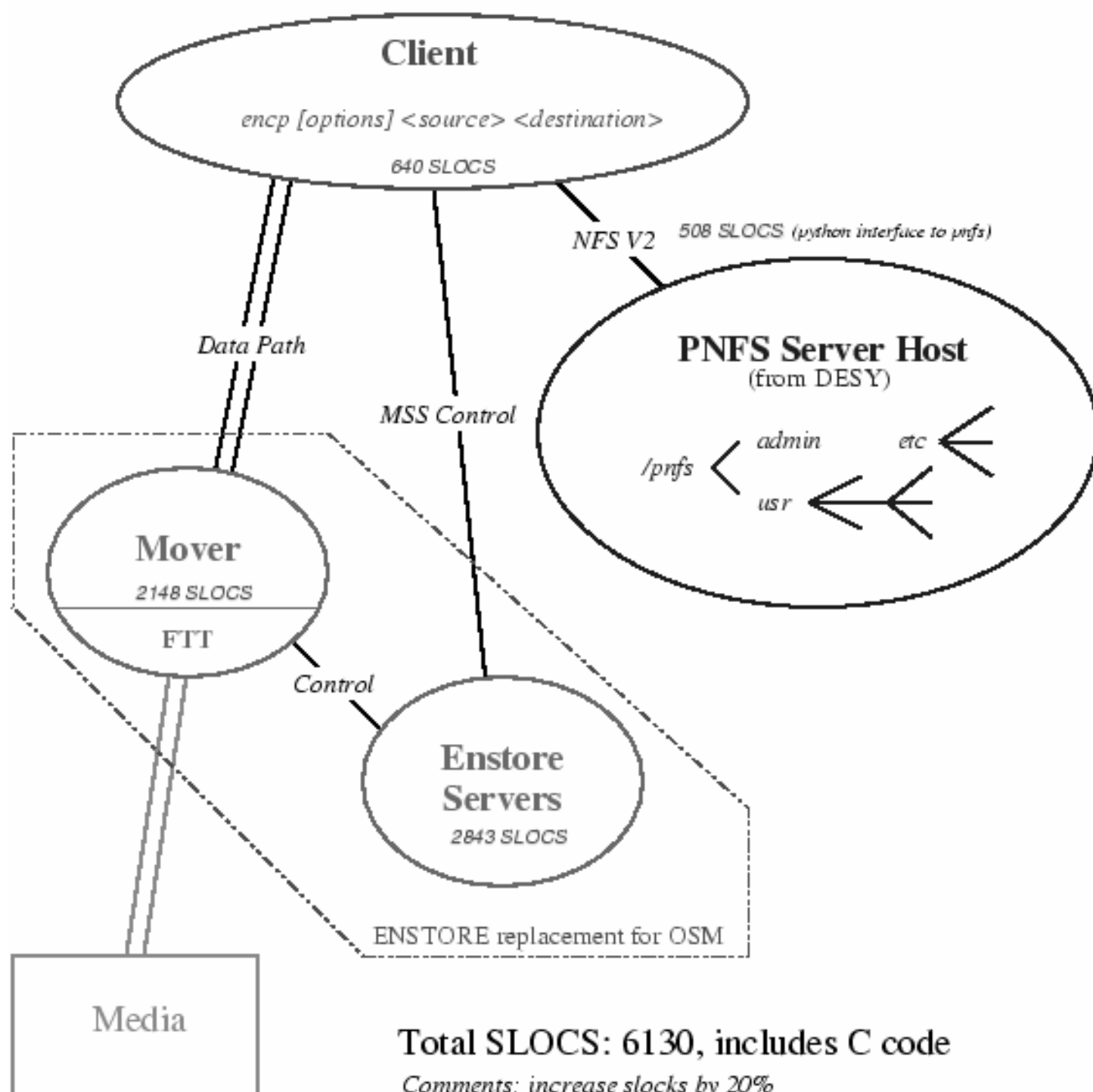
Feasible to replace OSM parts with our code

Decided to create a DESYish prototype at Fermilab
called it Enstore

ENSTORE TEST SYSTEM



Enstore at Fermilab



January–March Prototype Investigations:

Questions:

- Could we take Christmas code and build a prototype with most DESY features?
- Could the servers withstand heavy loading on a distributed system?

What we did:

- Built PNFS and installed PNFS
- Ported from Linux to AIX and IRIX
- Design implemented in multi–node environment
 - 6 Servers, mover, user copy program
- Implemented wrapping structure
- Integrated Fermilab FTT as a flexible low level tape driver (redwood and exabyte)
- Added database support (libtp) for 1 million files and 16 thousand volumes

Test system:

- STK 4400 silo with 2 redwoods
- Exabyte 8500
- Simulated "drives"
- $\frac{1}{4}$ speed fibre channel between 2 AIX nodes
- 56 Kb/S slow connection
- Repeatedly wrote eight 50 GB redwood tapes
- Ran system overnight with 18 end–user clients
- Data in == Data out

Results:

- DESY's basic design understood

- Python is a usable language

- some debugger issues

- Actual data movement in C

- Ready to work on rate and network tuning

- Ready to work on error recovery

- Able to reuse PNFS as distributed from DESY

- Our implementation of DESY design is sound

- Fermilab's knowledge of tape drives can be reused

- Confident that we can plan and finishing this project

DESY Workshop

Attended by CDF, D0, CD and DESY

DESY presented their system to us

- allowed wider audience to make sure nothing was missed or overlooked

Fermilab presented our implementation

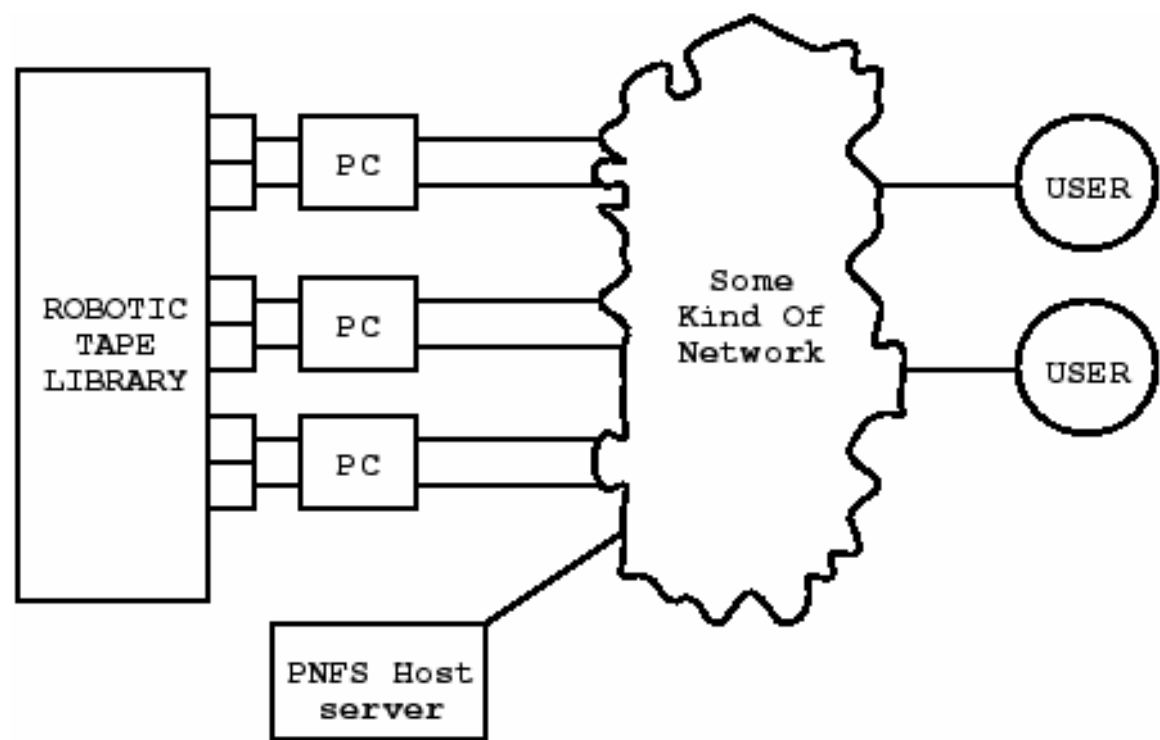
- allowed early critique of design before experts

Discussion

- DESY endorsed our implementation

- gave comments for improvement

- Collaboration possibilities



Hardware Components

DESY design and our investigations have led us to
networked computers and drives

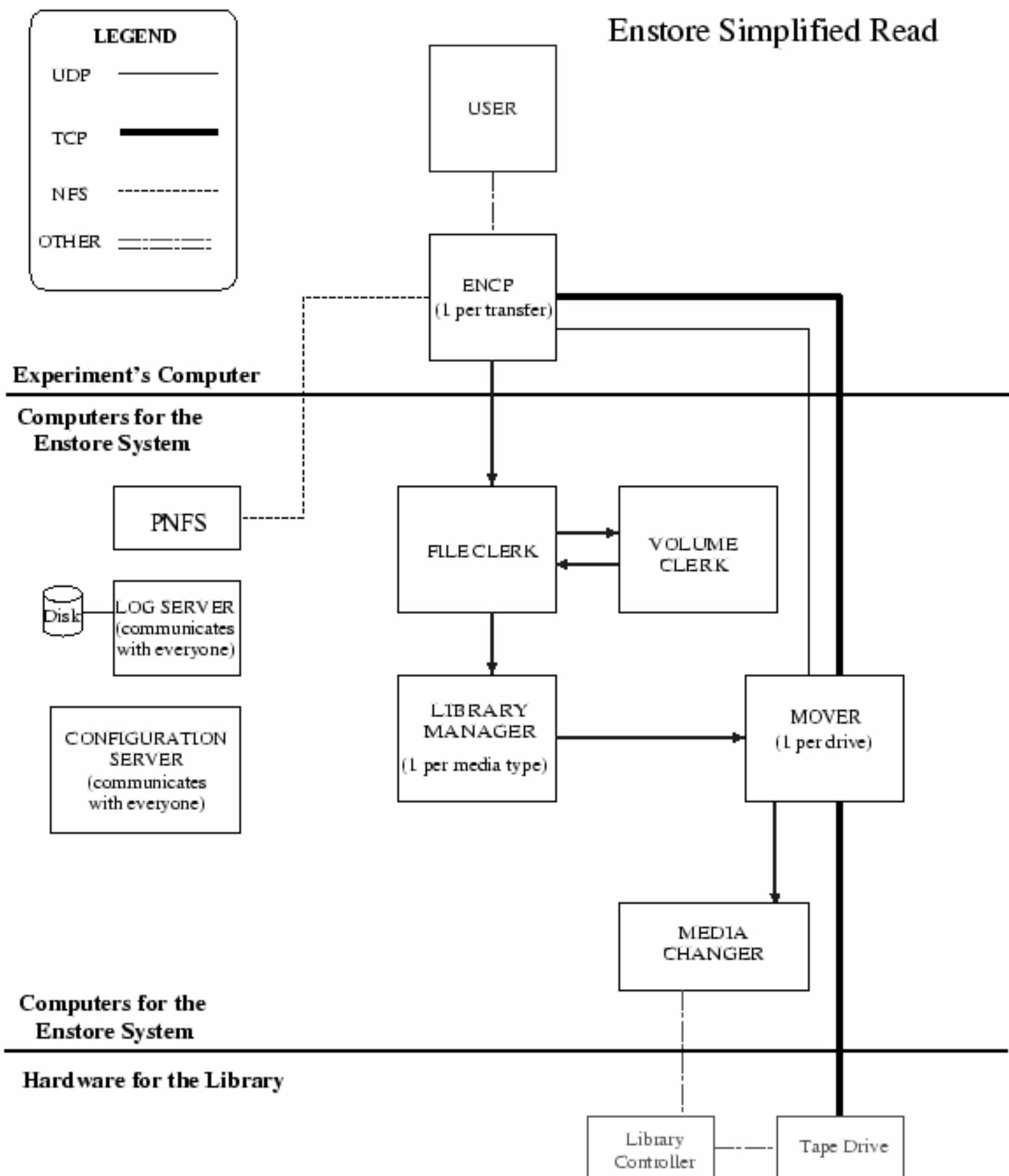
High level library controllers are part of design
low level controllers are not being investigated

Scaleable to Run II rates and sizes

Locally attached drives also possible

Code is portable – has run on AIX, IRIX, Linux

Committed to SCSI level access to media



Technical Description of Enstore Architecture

Design Philosophy: simple unthreaded code

Client/Server Architecture

- Reused Python server framework

Communication Protocols

- UDP for control

 - Messages fit in 1 datagram

 - Retries, unique ID

 - Clients can not hang servers

- TCP for data transfers

Fermilab FTT – portable hooks for dealing with tape drives

Supports networked distributed drive access

Databases

- Two tables (volume and file) required

- Currently using BSDdb for database support

 - Provides easy test setup with no licensing issues

 - Simple and fast enough for tests

 - Switching to LIBTP, also free for non-commercial

- Long term – choose most appropriate database

 - Run II database selection needs to be made

Software Components Outline

Pnfs

Encp

Servers and Clients

- Configuration server

- Library managers

- File clerks

- Volume clerks

- Media changers

- Log server

Mover

FTT

Administrative tools

Error control

- End-to-end

- Automatic retries

PNFS

Written and supported by DESY

Conceptually independent of the rest of the system

Particular implementation of a catalog to access MSS files

- Does not replace the experiments file catalogs

- Separable component

Clients:

- Looks like a file system, mounted like NFS

- Users can use standard unix commands (ls, mv...)

- No reading or writing of the file contents

Servers:

- DESY supplies nfsd and mountd

- Uses databases not files

Tests:

- Installed PNFS on 200 MHz Pentium Pro
scsi disks and 32 MB memory

- Loaded HPSS +Unitree namespace (250K names)

- Did name lookup on several machines and many
processes

- Each process returned 3–15 names/sec

- No errors in PNFS

PNFS Features:

Layers:

- 8 independent metadata records for each file

- We use layers to store:

 - Bit file ID

 - Debugging history

- Writes produce layer information, used to fetch files

Tags:

- Arbitrary number of metadata for each directory

- Inherited from parent directory by default

- We have tags that specify:

 - File family

 - File family width

 - Library to write to

 - Wrapper type (not implemented)

- Experiments administer tags, used when writing files

Wormholes:

- Hidden configuration and administrative files

ENCP

Simple program to copy files between media and user disk

Replacement for DESY OSMCP

Behaves like Unix cp command

Automatic retries where salient

Can not hang shared enstore servers

Reads:

Gets the bit file ID from PNFS

Submits bit file ID to file clerk

Enstore sets up file transfer

Read data from mover on TCP socket & write to disk

Writes:

Get family, width, library, wrappering tags in directory

Submit info to library manager

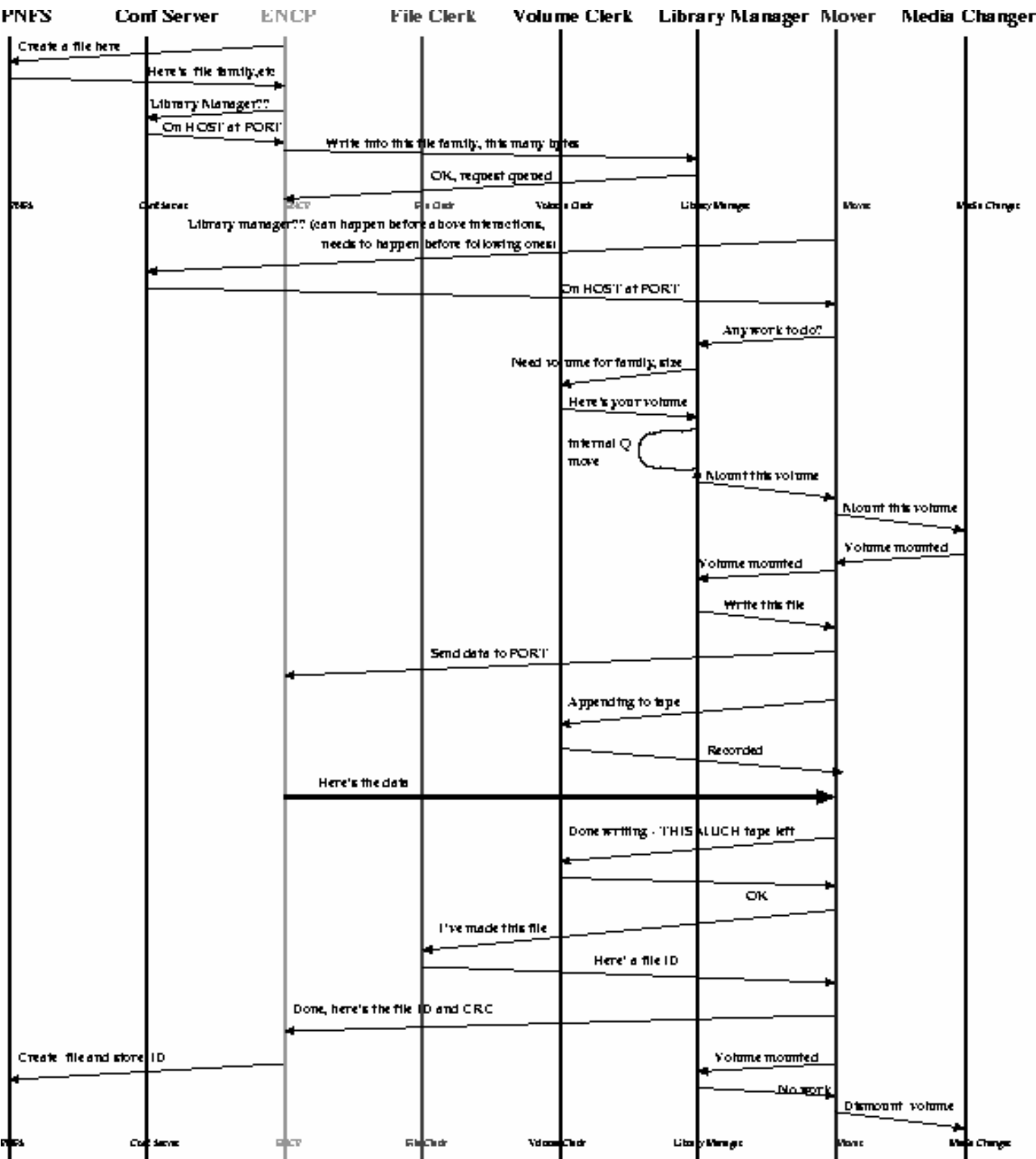
Enstore sets up file transfer

Read data from disk and write to mover on TCP socket

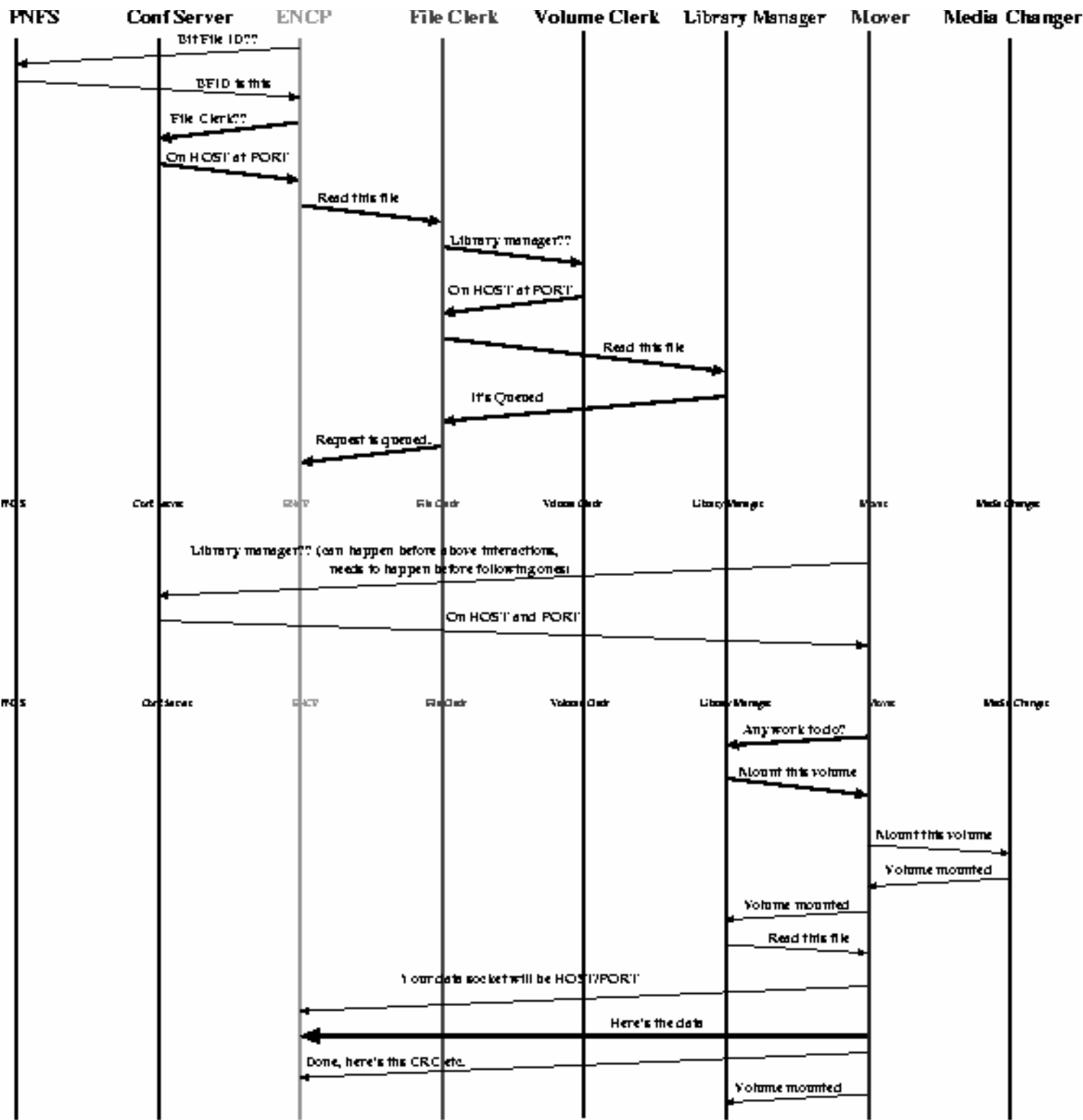
Investigating

Writing many files to 1 volume

Reading Some/all files from 1 volume



File Write Communications



File Read Communications

Configuration Server

Contains only well-known port in system

Maintains system configuration information:

- contains host and port for every server

- other configurable service specific parameters

Example of Redwood Mover configuration

`configdict[stk0.mover] =`

```
device:          /dev/rmt0
library:         redwood50.library_manager
driver:          FTTDriver
library_device:  0,0,9,1
media_changer:  STK.media_changer
logname:         STK0MOV'
```

Configuration can be queried

Configuration can be updated dynamically.

- All servers read configuration on startup

- Movers read configuration when idle

Library Manager

Main job is to submit the transfer requests to its movers

- Waits for mover to contact it and gives it request

- Tells movers which volume to mount

 - Writes – from file family or draw a blank

 - Reads – salient volume

- Serializes independent requests on same volume

- Transfers prioritized based on users flexible criteria

- Checks if the "width" of volumes already active

Maintains requests in 3 lists:

- Pending, unscheduled transfers

- Mounts in progress

- Transfers in progress

- Rudimentary display of lists implemented in prototype

Provides prioritized write access for DAQ systems

Investigations:

- Recover file family width should mover crash while writing

- Handle similar media of differing capacity

- Implement one appropriate for vaulted volumes

File Clerk

Maintains one keyed record for each file in the system:

- Bit file ID (primary key)

- Volume name where file was written

- Device specific file "spacing" information

- CRC of the file

 - Stored during writes

 - Checked during reads

- Sanity CRC

 - quick check to make sure we have requested file

Volume Clerk

Maintains one record for each volume in the system:

- Volume name (primary key) – external label

- File family name

- Wrapper format of the files

- Library

- Media type

- Time declared

- Time of first write

- Time of last access

- Remaining capacity

- Blocksize

- Info on how to space to end of data

- Write/read error and access counts

- User and System access level for volume

 - Read-only

 - Read/write

 - No access

 - Being written to

Media Changer

Mount/dismounts requested media by talking to library specific control micro

Expect library controller to be high level software

May serve multiple drives and library managers

Investigations:

Need on-demand operator mount media changer

Possibly understand scsi-level robot control

Log Server

Receives and logs messages from other processes to a file

Clients do not retry on UDP messages

System can not hang because of full logfile

Investigations:

Trace tool planned to aid in debugging

Mover

As name implies,

- Mover reads files from tapes and sends data to user

- Mover reads data from user and writes file to tape

Prototype mover used TCP sockets to send/receive data from the user

Each tape drive has its own mover process

- One computer can run many movers

Steps in transfer:

- When idle, asks library manager for work

- Library manager tells mover to mount volume x

- Mover calls media changer to mount volume

- Library manager gives mover transfer requests

- Mover contacts user

- Mover transfers requested data to user

To prevent system downtime, allow hot swapping of hardware and software component

- Mover crashes have minimal system impact

- Mover computers can rejoin without restarting system

If library is partitioned, one can dynamically assign movers to each partition to allocate resource where they are needed

User can specify a file structuring method (wrapping)

- Prototype implemented cpio

- Ansi planned

- Other formats as needed

Investigations:

- Early check before mount to make sure user still there

- Locally attached drives

- Implement error control

FTT

Embodies Fermilab's knowledge about tape hardware

Existing software

Table driven – method to add new tape drives

Provides MT and raw scsi access to tapes

Supports multiple types of serial media

Portable implementation

Parts of FTT that Enstore uses:

- Position media to correct file

 - By filemarks

 - By partitions

- Read/write data

- Write filemarks, both buffered and unbuffered

- Get remaining capacity of tape

- Python interface written

- Drive statistics

Administrative Tasks

Drain/halt requests for system maintenance

Database reports

- List all files and their metadata on particular volume

- List all volumes and their metadata in a particular family

Database Activities, without system disruption

- Insert volumes (blank or user generated)

- Remove exported volumes

- Switch ejected media to shelf library manager

- Mark volumes read-only

- Metadata consistency check – reap widows

- Empty trashbin and reclaim volumes

- Backups

Drive, robot and volume preventive maintenance statistics

Health and status monitor

User transfer request status

Error Control

Simplified because the user's request state is stored in encp

Encp can retry reads and writes if appropriate

- Retries are given a high priority

- Retries can flag that requests go to a different drive

Media and I/O errors are handled by the mover

- If trouble during load/unload

 - media jammed

- If several drives can't write volume

 - volume is disabled (at system level) for writes

- If a drive has errors on consecutive volumes

 - drive is disabled

- If volume has many recent errors

 - volume is disabled (at system level)

Alarm when critical work can't be done

Alarm when capacity of system falls below critical level

Much work still needs to be done to get correct end-to-end error control working in the prototype.

- Area where most lines of code would be added

Enstore Design Philosophy

System should not try to solve all MSS problems, but rather focus on a system that accesses the large Run II datasets

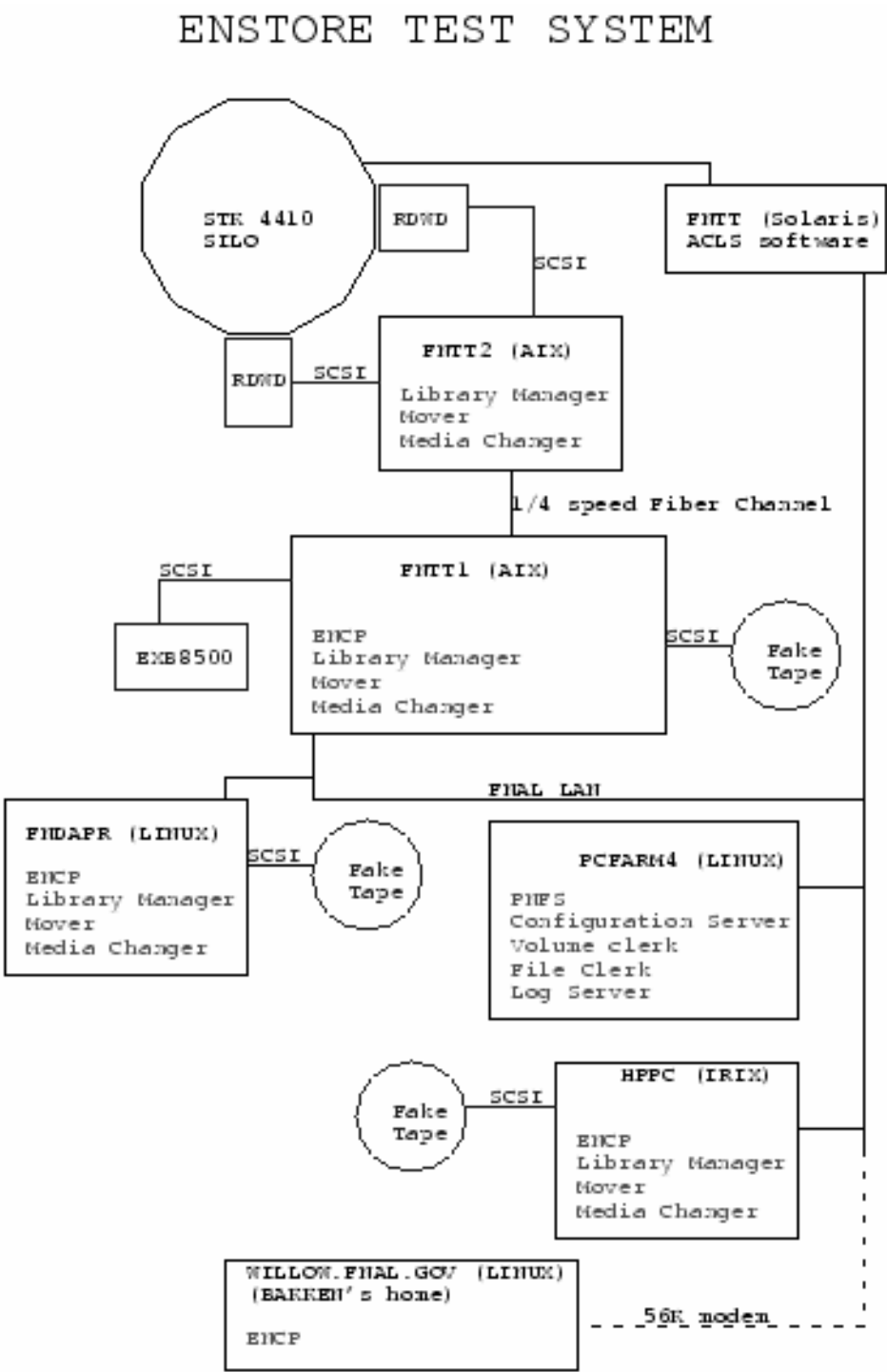
System should never trade uptime for features

System must be inherently scaleable to the expected Run II rates and capacity

System must be small and nimble in design so it is capable of satisfying changing requirements

System should reuse components where ever possible to take advantage of lessons learned in the past

System should support well-defined more primitive access



100 BULLET ANALYSIS

HPSS Workshop Outcome

Document what Run II needs from MSS

Analyze how/if HPSS could meet requirements

Result was document, authored by Fermilab and IBM
edited by IBM, so some IBM biases remain

100 Bullet List == Extracted list of "shalls" from document

List was grouped into Categories

1. Retrieve and Specify Tape Contents
2. Eject to Shelf
3. Read Only
4. Import
5. Open Format
6. Resource Management
7. Availability
8. Fault Tolerance
9. Administration and Monitoring
10. Data Migration
11. Unclassified

Enstore was asked to look at each bullet and determine how it would fulfill the requirement.

Retrieve and Specify Tape Contents

Requirements:

Data files shall be located on tape to optimize access.

Volumes shall only contain files of the same grouping.

Media shall be efficiently used.

General purpose tools are needed to generate reports of what is on tape and associated attributes.

Enstore:

Puts files on tape in the order the files were submitted using a file family and file family width scheme.

Expects ftt to provide correct information from the media devices and will use this information to fill tapes.

Reporting tools need to be developed.

Enstore decided not to span files across tapes.

Eject to Shelf

Requirements:

Data shall be written so volumes can be ejected from the robot to shelf storage

Ejection must not affect system performance

Utilities/reports needed to aid in ejecting volumes

Need to be able to specify behavior of requests for ejected volumes

Metadata must allow utility programs to read data.

Enstore:

Ejection to shelf is part of design – it means just changing the library manager that controls the media.

If experiments can order tapes appropriately, enstore will not impede ejecting them to the shelf.

Wrappering of files provides self–describing tapes

Reporting tools and other utilities need to be developed.

Read Only

Requirements:

Users must be able to designate tapes as read-only when they deem appropriate

Enstore:

2 flags are available, one that the system controls and one that the user controls. If the flag is set to read-only, tapes can only be read.

Import

Requirements:

It should be possible to import tapes into the system without copying the data if the metadata is available

It should be possible to export the tapes and metadata from the system to other sites without copying

Enstore:

Since the volumes in the enstore library are self-describing, it should be relatively easy to write a utility to read the metadata and import the volumes or conversely to write the metadata so other programs can read the tapes outside the system

Open Format

Requirements:

Tape format must be known

Utilities must be available to read tapes outside system

Enstore:

Tape is self-describing with each file wrapped
currently cpio wrapping is used

More wrappers should be available and specified by
users

Unix tools are available to read wrapped files

Resource Management

Requirements:

Drives, robot arm, etc. in short supply & must be managed properly to match changing rates and conditions

Tape mounts must be minimized to increase performance

Need priority mechanism to specify which request are more important than others

Need ability to write directly to tape

Quotas are needed

User's should not have alot of "baggage" software on their system

Users need API to control file transfers

Users need estimate of time-to-complete

Controls need to be in place to prevent overloading

Resource Management

Enstore:

Movers, and hence drives, can be dynamically assigned to match the conditions and priorities

Once tape is mounted, the queue is checked & all requests for volume are done before dismounting.

User can specify a priority – this can be developed as much as is needed.

Users can write to tape bypassing disk via movers

Simple quotas in place – can be developed as much as necessary

User's baggage – maybe none, at most it will be the standard Fermilab packages like fft, python...

API can be provided for users

Time-to-complete estimate is very difficult to provide

Overloading controlled by arbitrating for sockets to limit number of concurrent transfers. Pnfs namespace separate from data transfers.

Availability

Requirements:

Unattended operation required

Uptime = continuous for 30 days

Must be able to sustain rate

Need alarms when fall below threshold

Enstore:

Unattended operation, uptime requirements, sustain rates are all goals, not are proven by enstore today.

Substantial error analysis and testing planned

Servers will be robust

Dynamic configuration allows flexibility

Alarms – straightforward development. Much info already logged, need to extract and monitor

Fault Tolerance

Requirements:

Single point failures shall not cause system failure

Fail-over capabilities must be part of system

Retries, where appropriate, should be automatic

Bad components should be taken offline and flagged and the transfers should continue at reduced rate

Enstore:

Robust against mover/drive failures because system is distributed.

Single point of failure servers are simple and very robust.

Extensive end-to-end error control and retry development in progress.

Administration and Monitoring

Requirements:

Gui/CLI interfaces needed for administration

Trashbin needed for recovery

Need Health/Monitoring Capabilities

Drive/Disk/Transfer statistics need to be available and monitored

Enstore:

CLI interfaces are primary interface

Trashbin via pnfs

Statistics – probably web based for current work and transfer history

Data Migration (HSM)

Requirements:

Need HSM capabilities (migration) to handle all levels hierarchy, shelf, tape, cache disk, and user disk

Enstore:

Enstore only attempts to handle the shelf and tape levels and expects the Run II data access to manage the rest.

A cache disk is a planned upgrade if we collaborate with DESy

Unclassified

Requirements:

Aid in evaluating drives, media, robots...

Be scaleable so one can adapt to changes

Enstore:

Layered approach – ftt contains Fermilab's tape knowledge and can be adapted for new drives. Changes in movers may be required to take full advantage of unusual circumstances

Distributed movers are very scaleable. The software will not impede the hardware.

Implementation Plan

Goal is to have a working system throughout development

Start with prototype which represents 6–8 MM effort

Evolve to the Run II release of Enstore

Simultaneous development possible on many fronts

Ongoing integration with RIP and Data Handling

Note that DESY's system evolved from OSM to their current system as well.

Effort	Phase
4	Organization and Methods of Working
12	Requirements, CDF/D0 Data Access, RIP...
84	Evolution of Prototype to Run II Product
28	Integration, Commissioning and Tuning

128 **MW** Total to Complete

Total effort for a Run II Release of Enstore is 2.5 MY

Applying the Run II scaling factor of 0.7, the total project effort for a Run II Release of Enstore is 3.5 FTE years.

Must be substantially finished May 99 == 9 months before beginning of Run II

Comparisons:

How does the 2.5 MY for Enstore compare to other projects of similar scope which were recently completed?

SDSS:	Astroline:	1.5 MY
	Archiver:	1.5 MY

UPS/UPD	3 MY
---------	------

Dart Data Logger	0.5 MY
------------------	--------

Assumption:

The prototype system should also be maintained and developed as the project proceeds as a demonstration platform that will allow developers of higher level services and users to provide your team rapid feedback on the suitability of the system as it is being created.

Implementation Plan Details

Organization

4 MW

- Packaging methods
- Coding standards
- Development tools
- Bug reporting and tracking procedure

Requirements

12 MW

- Input and specification from experiments
- Input and specification from mss groups and operators
- Understand commonality of requirements and iterate
- Understand interfaces to other Run II Projects such as
Data Access, Production Management and RIP
- Understand testing dates and scope
- Understand hardware constraints
- Understand testing criteria and dates
- Agree on change control mechanisms

Implementation Plan Details

Evolution and Construction

84 MW

- Client server framework

 - Communications protocol and errors

 - Robustness

- Error handling philosophy

 - Component Retries

 - End-to-end recovery

 - Fault tolerance and availability

 - Reliability

- Encp framework

 - Design evaluation

 - Options and switch analysis

 - Optimization

 - Bypassing system – direct calling of movers

 - Binary distribution studies

- Improvements to Servers/Clients and Clerks Design

 - Configuration server and clients

 - Library manager and clients

 - Media Changer and clients

 - Volume clerk and clients

 - File clerk and clients

 - Log server and clients

- Mover Modifications

 - File wrappering – self describing, different types

 - Optimization

 - FTT – new drives to support

Testing Framework

- Debug and integration framework

- Configure Test Hardware Platform

Database framework

- Evaluation of underlying database choice

- User Queries

- Fault tolerance

- Backup

Admin tools

- Pnfs

- Web status

- User queries and reports

- System and Tape Monitoring and Statistics

Fmss interface and compatibility

Disk Cache (DESY cooperation)

Data Integrity, with respect to Fermilab Policy

- Data protection, Authentication, Access

- Accidents

Direct access to devices (bypassing system)

Integration

28 MW

- Integration with Experiment RIP and Production Farms

- Integration with Experiment Data Handling and Analysis

- Commissioning

- Tuning